

Webcam Motion Detection Application: Development Plan

Document type	Software Development Plan
Version	1.0 (Draft for client review)
Date	October 4, 2026
Companion document	System Architecture Document v1.0
Status	Proposed

1. Introduction

1.1 Purpose

This plan explains **how** the Webcam Motion Detection Application will be built, tested, delivered and handed over. It turns the System Architecture Document into concrete phases, tasks, deliverables, estimates and quality gates.

1.2 Product Summary

A web application that opens the user's webcam and shows the live feed in **black and white while the scene is still**. When **movement is detected**, the feed switches to **full color** and an **alert message** is shown (with optional sound and browser notification). Motion events are logged and sensitivity is adjustable.

1.3 Objectives

1. Deliver a working MVP quickly, so the client can see the core behavior early.
2. Keep detection accurate (few false alerts) and fast (under 300 ms to react).
3. Keep all video processing on the user's device for privacy.
4. Build modular code so features such as a backend or multi-camera support can be added later without a rewrite.

1.4 Scope Summary

Phase	In scope
MVP (Phases 1–3)	Camera access, black-and-white/color switching, motion detection, alert banner and sound, start/pause/stop, sensitivity control
Release 1.0 (Phases 4–5)	State machine hardening, event log with snapshots, browser notifications, settings persistence, accessibility, cross-browser support, deployment
Optional Phase	Backend: user accounts, event sync, email/SMS/push alerts

Future	Multi-camera, region-of-interest drawing, ML person detection, video clip recording
---------------	---

2. Development Approach

2.1 Methodology

Agile (Scrum-lite) with 1-week sprints. A one-week cycle suits a small project: the client sees a working increment every Friday and can change direction quickly.

Ceremony	When	Purpose
Sprint planning	Monday, 30 min	Pick tasks for the week
Daily check-in	Daily, 10 min (or async chat)	Share progress and blockers
Sprint demo	Friday, 30 min	Show working software to the client
Retrospective	Friday, 15 min	Improve the process

2.2 Guiding Principles

- **Vertical slices first:** get camera → detection → visual change working end to end, then improve each part.
- **Pure logic is separated from the browser:** detection and state-machine code has no DOM access, so it is easy to unit test.
- **Make it work, make it right, make it fast,** in that order.
- **Test with real-world video:** still room, walking person, lighting changes.
- **Privacy by default:** no frames are sent anywhere in Phase 1–5.

2.3 Roles and Responsibilities

Role	Responsibility	Effort (typical)
Project Manager / Product Owner	Requirements, priorities, client communication, acceptance	Part-time
Frontend / Full-stack Developer(s)	Implementation of all modules	Full-time
UI/UX Designer	Wireframes, visual design, accessibility review	Part-time (Phases 1–3)
QA Engineer	Test plan, manual and automated tests, cross-browser checks	Part-time, full-time near release
DevOps (shared)	CI/CD, hosting, monitoring	Few days

Client	Provide requirements, review demos, approve milestones	Weekly
--------	--	--------

Solo developer? One person can cover all roles. Add roughly 30–40% to the timeline in Section 5.

3. Development Environment and Tooling

Area	Choice
Language	TypeScript
Framework	React 18 + Vite (or vanilla TS if the client wants minimal dependencies)
Styling	CSS Modules or Tailwind CSS
State/events	Lightweight typed event bus (custom, ~40 lines)
Storage	localStorage (settings), IndexedDB via idb (events, snapshots)
Linting / format	ESLint + Prettier + EditorConfig
Unit testing	Vitest
End-to-end testing	Playwright with Chrome fake camera
Version control	Git (GitHub or GitLab), trunk-based with short-lived feature branches
CI/CD	GitHub Actions or GitLab CI
Hosting	Netlify / Vercel / Cloudflare Pages (static HTTPS)
Task tracking	Jira, Trello, Linear or GitHub Projects
Documentation	Markdown in repo (/docs), README, ADRs for key decisions

3.1 Local Setup (for developers)

1. Create the project from a standard React + TypeScript starter template.
2. Add the small set of supporting libraries: IndexedDB helper, unit-testing framework, linting/formatting tools and the end-to-end testing tool.
3. Run the app locally. Browsers allow camera access on **localhost** without HTTPS.

Browsers allow camera access only on **HTTPS** or **localhost**. To test on a phone on the local network, use a secure tunnel service or a local HTTPS setup.

3.2 Branching and Commit Rules

- main is always deployable. Work happens on feature/- branches.
- Pull requests need **1 review** and **passing CI** (lint, tests, build).

- Commit messages follow Conventional Commits: feat:, fix:, test:, docs:, refactor:.

3.3 Definition of Done (per task)

- ☐ Code reviewed and merged via pull request
- ☐ Unit tests written and passing (for logic modules)
- ☐ No ESLint errors; formatted with Prettier
- ☐ Works in Chrome and one other browser
- ☐ Accessibility basics checked (keyboard, labels)
- ☐ Documentation or code comments updated
- ☐ Demonstrated or verified against acceptance criteria

4. Work Breakdown Structure (WBS)

Phase 0: Discovery and Design (Week 1)

ID	Task	Output
0.1	Kick-off meeting, confirm scope and priorities	Meeting notes
0.2	Confirm open questions from architecture doc (web vs desktop, 24/7 use, alert types, storage, privacy)	Signed-off requirements
0.3	Create wireframes (main screen, settings, event list, permission-denied screen)	Figma/PNG wireframes
0.4	Choose visual style (colors, typography, alert design)	Style guide
0.5	Technical spike: confirm detection works on a laptop webcam with a 50-line prototype	Prototype and findings
0.6	Set up repository, CI, project board	Repo with CI green
0.7	Finalize test plan outline and acceptance criteria	Test plan v0.1

Exit criteria: Client approves requirements and wireframes; spike proves feasibility.

Phase 1: Camera and Visual Foundation (Week 2)

ID	Task	Details	Est.
1.1	Project scaffold	Folder structure from architecture doc, ESLint, Prettier, Vitest	0.5d
1.2	Event bus	Typed emit/on/off with unit tests	0.5

			d
1.3	Camera service	getUserMedia, start/stop, error mapping (denied, not found, in use)	1.5 d
1.4	Video panel component		0.5 d
1.5	Render controller	Toggle grayscale(100%) with 200 ms CSS transition, manual test toggle button	0.5 d
1.6	Permission and error screens	Denied / no camera / retry	1 d
1.7	Start / Stop controls	Basic UI and status badge	0.5 d

Demo: Open the app, grant permission, see live video in black and white; a debug button toggles color.

Phase 2: Motion Detection Core (Weeks 3–4)

ID	Task	Details	Est.
2.1	Frame preprocessor	Hidden canvas 160×120, getImageData, grayscale conversion, reusable buffers	1.5 d
2.2	Optional blur	3×3 box blur to reduce noise	0.5 d
2.3	Motion detector	Per-pixel diff, threshold, motion ratio, adaptive background	2 d
2.4	Frame loop	Time-gated loop at ~15 FPS using requestAnimationFrame with fallback	1 d
2.5	Connect to render	Motion → color, no motion → black and white (simple version, no debounce yet)	0.5 d
2.6	Alert banner (basic)	Red banner with timestamp, aria-live="assertive"	1 d
2.7	Sensitivity slider	Map 1–10 to pixel and area thresholds	1 d
2.8	Unit tests for detector	Synthetic frames: identical, small change, large change, uniform brightness shift	1.5 d

2.9	Debug overlay (dev only)	Show motion ratio, FPS, changed-pixel mask	1 d
-----	--------------------------	--	-----

Demo (MVP): Wave a hand → feed turns to color, banner appears. Stay still → black and white.

Phase 3: Reliable Behavior (Week 5)

ID	Task	Details	Est.
3.1	Motion state machine	STOPPED → IDLE → CANDIDATE → MOTION → COOLDOWN with confirm frames and cooldown timer	2 d
3.2	State machine tests	Table-driven tests for every transition, timers via fake timers	1 d
3.3	Integrate state machine	Render and alerts now driven by state events, not raw detection	0.5 d
3.4	Alert manager	Banner + Web Audio beep + rate limiter; sound unlock on first click	1.5 d
3.5	Pause / resume	Keeps camera on but stops detection	0.5 d
3.6	Cooldown setting	UI control and persistence hook	0.5 d

Demo: No flicker, no alert on single noisy frame; color persists a few seconds after motion stops, then fades back to black and white.

Phase 4: Features and Persistence (Weeks 6–7)

ID	Task	Details	Est.
4.1	Settings manager	Validate, persist to localStorage, broadcast SETTINGS_CHANGED	1 d
4.2	Settings panel UI	Sensitivity, cooldown, alert toggles, snapshot toggle	1.5 d
4.3	Event logger	IndexedDB store, create/close events, cap at 500	1.5 d
4.4	Snapshot capture	Grab frame on MOTION_STARTED to a compressed JPEG blob	1 d
4.5	Event list UI	Time, duration, peak score, thumbnail; clear-all and delete	1.5 d

4.6	Browser notifications	Permission flow and fallback when blocked	1 d
4.7	Robustness	Brightness-shift filter, camera-disconnect recovery (track.onended), tab-visibility handling	2 d
4.8	Web Worker move	Move preprocessing and detection into a worker with OffscreenCanvas (fallback to main thread if unsupported)	2 d
4.9	Optional ROI	Rectangle selection to ignore parts of the frame	1.5 d (optional)

Demo: Full feature set; settings survive reload; events listed with snapshots.

Phase 5: Quality, Polish and Release (Weeks 8–9)

ID	Task	Details	Est.
5.1	UI polish	Responsive layout, final visuals, animations, empty states	2 d
5.2	Accessibility audit	Keyboard, screen reader, contrast, reduced-motion	1.5 d
5.3	Cross-browser testing	Chrome, Edge, Firefox, Safari, Android Chrome, iOS Safari	2 d
5.4	Performance tuning	Profile on a low-end laptop; adjust resolution/FPS defaults	1.5 d
5.5	End-to-end tests	Playwright with fake camera video files (still, motion)	2 d
5.6	Accuracy test run	Measure false positives/negatives on recorded clips	1 d
5.7	Security and privacy review	CSP, HTTPS/HSTS, privacy notice, no frame leaves device	1 d
5.8	Documentation	README, user guide, admin/deployment guide, architecture updates	1.5 d
5.9	Deployment	Production hosting, domain, TLS, CI/CD to staging and production	1 d
5.10	UAT with client	Client tests, bug fixes, sign-off	3 d
5.11	Release 1.0 and handover	Tag, release notes, training session	1 d

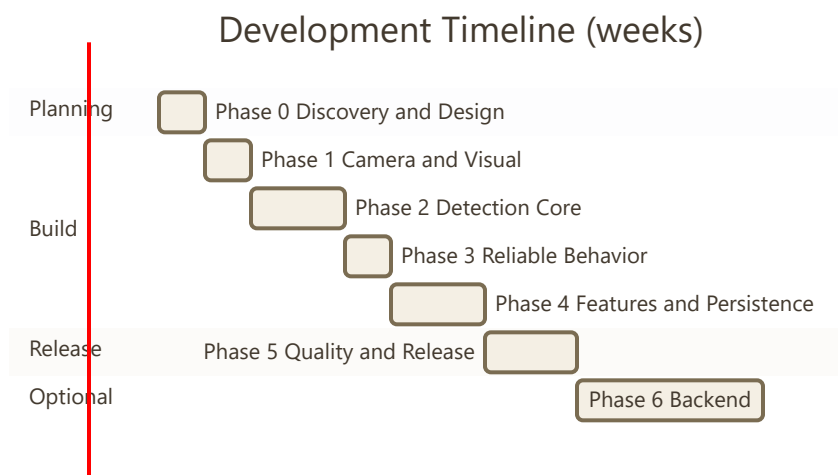
Exit criteria: All acceptance criteria in Section 8 met; no open critical or high bugs.

Phase 6: Optional Backend (Weeks 10–13)

ID	Task	Details	Est.
6.1	Backend design	API spec (OpenAPI), database schema, auth model	2 d
6.2	Project setup	Node.js (Fastify) or FastAPI, Docker, CI	1 d
6.3	Authentication	Sign-up/login, JWT, password reset	3 d
6.4	Events API	POST /api/events, GET /api/events, snapshot upload to object storage	3 d
6.5	Settings sync	GET/PUT /api/settings	1 d
6.6	Notification service	Queue worker for email (SES/SendGrid), SMS (Twilio), web push	4 d
6.7	Client sync service	Opt-in upload, offline queue, retry with backoff	3 d
6.8	Dashboard	Event history page with filters	3 d
6.9	Backend security	Rate limiting, input validation, retention policy	2 d
6.10	Tests and deployment	API tests, load test, infrastructure as code	3 d

5. Timeline and Milestones

5.1 Schedule (one full-time developer plus part-time QA/design)



Dates are illustrative. Replace the start date with the actual kick-off date.

5.2 Milestones

#	Milestone	Target	Client sign-off
---	-----------	--------	-----------------

M0	Requirements and wireframes approved	End of Week 1	Yes
M1	Visual foundation demo (camera + black-and-white)	End of Week 2	Review
M2	MVP demo (motion → color + alert)	End of Week 4	Yes
M3	Stable behavior (state machine, sound)	End of Week 5	Review
M4	Feature complete (log, settings, notifications)	End of Week 7	Yes
M5	Release 1.0 (UAT passed, deployed)	End of Week 9	Yes
M6	Backend release (optional)	End of Week 13	Yes

5.3 Effort Summary

Phase	Duration	Approx. person-days
0. Discovery and Design	1 week	5–6
1. Camera and Visual	1 week	5
2. Detection Core	2 weeks	10
3. Reliable Behavior	1 week	6
4. Features and Persistence	2 weeks	11–13
5. Quality and Release	2 weeks	18
Total for Release 1.0	~9 weeks	~55–60
6. Optional Backend	4 weeks	~25

Add a **15–20% contingency buffer** for unknowns, such as device-specific camera quirks.

6. Module Implementation Approach

This section describes **what each module must do and how it should behave**, to guide the developers during Phases 1–4.

6.1 Camera Service

- Ask the browser for the webcam at a moderate resolution (about 640 × 480, 30 FPS ideal) with no audio.
- Attach the stream to the video element and start playback.
- Translate browser errors into four clear categories: permission denied, no camera found, camera in use by another app, and unknown error. Each category gets its own friendly message on screen.

Be sure to read the entire specification and follow the examples that have been provided to ensure that the

- Provide start and stop actions; stopping must release the camera so the browser's camera-in-use light turns off.

6.2 Black-and-White / Color Display

- The visible video is shown in grayscale by default using a simple visual styling effect, which is smooth and uses almost no processing power.
- When motion is confirmed, the grayscale effect is removed with a short (about 200 ms) fade, and a red outline highlights the video.
- When the motion state ends, the effect returns.
- A canvas-based approach is used only if the client later needs recordings or snapshots to be black and white.

6.3 Frame Preprocessing

- At about 15 times per second, copy the current video frame onto a small hidden image area (about 160×120 pixels).
- Convert the pixels to shades of gray.
- Optionally apply a light blur to reduce camera noise.
- Reuse the same memory buffers every frame to avoid slowdowns.

6.4 Motion Detection

- Compare each new frame with a slowly updating reference image of the "still" scene.
- Count the pixels that changed by more than a set amount; the share of changed pixels is the motion score.
- Motion is flagged when the score is above the area threshold.
- The reference image adapts slowly, so gradual lighting changes (clouds, sunset) are absorbed instead of triggering alerts.
- Frames with a sudden, scene-wide brightness shift (such as a light switch or camera auto-exposure) are treated carefully to avoid false alerts.

6.5 Motion State Machine

- Five states: Stopped, Idle, Candidate, Motion and Cooldown.
- **Idle** → **Candidate**: a single frame shows motion.
- **Candidate** → **Motion**: motion continues for a few frames in a row (debounce). The alert fires here.
- **Candidate** → **Idle**: motion disappears before confirmation (treated as noise).
- **Motion** → **Cooldown**: motion stops.
- **Cooldown** → **Idle**: no motion for the configured cooldown time (about 3 seconds); the display returns to black and white.
- **Cooldown** → **Motion**: motion resumes during the cooldown; the same event continues.

6.6 Alert Manager

- Show a red on-screen banner with the time of detection, announced to screen readers.
- Play a short beep. Browsers block sound until the user has interacted with the page, so audio is enabled when the user clicks Start.
- Send a browser notification if the user has allowed it; if blocked, banner and sound still work.
- Limit how often alerts can fire so the user is not flooded.

6.7 Event Logger and Snapshots

- When motion is confirmed, create an event record with start time and optionally a snapshot photo.
- When motion ends, record end time, duration and peak motion score.
- Keep the most recent 500 events; remove the oldest automatically.
- Video itself is never stored.

6.8 Settings

- Sensitivity (1–10), cooldown time, alert on/off switches and snapshot on/off.
- Values are validated, saved on the device and restored on the next visit.

6.9 Performance Approach

- Process only about 10–15 frames per second at small size.
- In Phase 4, move the heavy image work off the main page thread so the interface stays smooth.
- Lower the processing size automatically if the device cannot keep up.

6.10 Sensitivity Mapping (starting values)

Slider	Pixel change needed	Share of image that must change	Behavior
1 (low)	45	6%	Only large, obvious movement
3	35	4%	
5 (default)	25	2%	Balanced
7	18	1%	
10 (high)	12	0.4%	Very sensitive; more false alerts

These are starting points. They will be tuned in Phase 5 using recorded test clips.

7. Testing Plan

7.1 Test Levels

Level	Scope	Tools	Owner	When
Unit	Grayscale, detector, state machine, settings validation, event bus	Vitest	Developer	Every commit
Component	UI components (controls, banner, event list)	Vitest + Testing Library	Developer	Every commit
Integration	Frame pipeline with recorded video frames	Vitest + fixtures	Developer/ QA	Per sprint
End-to-end	Full flow with fake camera	Playwright	QA	Nightly and before release
Performance	FPS, CPU, memory over 30 min	DevTools profiler	QA	Phase 5
Cross-browser/ device	Browser and OS matrix	BrowserStack or real devices	QA	Phase 5
Accessibility	Keyboard, screen reader, contrast	axe, Lighthouse, manual	QA/ Designer	Phase 5
Security/ privacy	CSP, HTTPS, data flow review	Manual review	Developer/ QA	Phase 5
UAT	Real-world use by client	Test script	Client	Phase 5

7.2 Test Data

Record short clips (10–60 s) for the test fixtures:

1. Still room, stable light (expect **no motion**)
2. Still room with camera noise in low light
3. Person walks across frame
4. Small hand movement near the camera
5. Slow movement
6. Gradual lighting change (curtain opened)
7. Sudden light switch on/off
8. Screen or TV flicker in background

Automated tests can play these recorded clips through Chrome's built-in fake camera feature, so the full detection flow can be tested without a live person.

7.3 Key Test Cases

7.3 Key Test Cases

ID	Scenario	Expected result
TC-01	First visit, grant camera	Live video appears in black and white
TC-02	Deny camera permission	Friendly error and instructions
TC-03	No camera connected	"No camera detected" with retry
TC-04	Still scene for 60 s	0 alerts; stays black and white
TC-05	Person enters frame	Color and alert within 300 ms
TC-06	Movement stops	Stays color during cooldown, then returns to black and white
TC-07	Single-frame noise spike	No alert (debounce)
TC-08	Gradual lighting change	No alert
TC-09	Sudden light switch	No alert, or at most one alert, then stable
TC-10	Raise and lower sensitivity	Detection threshold changes accordingly
TC-11	Reload page	Settings preserved
TC-12	Unplug camera during use	Error shown; auto-reconnect when plugged back
TC-13	Switch to another tab and back	Monitoring resumes correctly
TC-14	Notifications blocked	Banner and sound still work
TC-15	500+ events logged	Oldest removed; no slowdown
TC-16	Keyboard-only use	All controls reachable and operable
TC-17	Screen reader	Alert announced
TC-18	30-minute run	No memory growth; FPS stable

7.4 Defect Management

Severity	Definition	Fix target
Critical	App unusable, camera never starts, false alerts constantly	Same day
High	Core behavior wrong (no color on motion, no alert)	Within sprint
Medium	Feature works but with notable issue	Next sprint
Low	Cosmetic	Backlog

Release rule: zero Critical or High open defects.

8. Acceptance Criteria

#	Criterion	How verified
AC-1	Feed is in black and white when the scene is still	TC-01, TC-04
AC-2	Feed turns to color within 300 ms of real movement	TC-05, measured in E2E
AC-3	An alert message appears when motion is detected	TC-05
AC-4	Feed returns to black and white after cooldown	TC-06
AC-5	0 false alerts in a 60-second still scene under stable lighting	TC-04
AC-6	Detection correct on at least 95% of recorded test clips	Accuracy run (task 5.6)
AC-7	Sensitivity and cooldown adjustable and remembered	TC-10, TC-11
AC-8	At least 15 FPS processing on a mid-range laptop, no UI lag	Performance test
AC-9	Works on latest Chrome, Edge, Firefox, Safari (desktop) and Android Chrome / iOS Safari	Cross-browser run
AC-10	No video leaves the device (Phase 1–5)	Network inspection
AC-11	Accessibility: keyboard operable, alert announced to screen readers	TC-16, TC-17

9. CI/CD and Release Process



9.1 Pipeline Steps

1. Install dependencies from the locked list
2. Check code style and quality (lint)
3. Run unit tests with coverage (fail below an agreed threshold, for example 80% on the detection and state-machine logic)
4. Build the production bundle
5. Run end-to-end tests (on merge to the main branch)
6. Deploy a preview (per pull request), staging (on merge) and production (after manual approval)

or deploy a preview (per pull request), staging (on merge), and production (after manual approval).

9.2 Environments

Environment	Purpose	URL (example)
Development	Local work	localhost:5173
Preview	Per-PR review	pr-123.motion-monitor.dev
Staging	QA and client demos	staging.motion-monitor.com
Production	Live	app.motion-monitor.com

9.3 Release Checklist

- ☐ All acceptance criteria verified
 - ☐ No open Critical/High bugs
 - ☐ Release notes written and version tagged (semantic versioning, e.g. v1.0.0)
 - ☐ Production HTTPS certificate valid; HSTS and CSP headers set
 - ☐ Privacy notice published
 - ☐ Rollback plan tested (redeploy previous build)
 - ☐ User guide and admin guide handed to client
 - ☐ Client sign-off recorded
-

10. Risk Management

#	Risk	Likelihood	Impact	Mitigation	Owner
R1	False alerts from lighting changes or camera auto-exposure	High	High	Adaptive background, brightness-shift filter, tuned thresholds, sensitivity control; test fixtures for lighting changes	Developer
R2	Poor performance on low-end devices	Medium	High	Small processing resolution, FPS cap, Web Worker, performance budget tests	Developer
R3	Browser differences (especially Safari/iOS)	Medium	Medium	Test early in Phase 1 spike, feature detection and fallbacks	QA
R4	Background tab throttling stops detection	High	Medium	Document the limitation; notifications; propose desktop/backend version if 24/7 is required	PM

R5	Audio blocked by autoplay policy	Medium	Low	Unlock audio on Start click; always show visual alert	Developer
R6	Scope creep (ML detection, multi-camera, recording)	Medium	High	Change-control process; park in backlog; estimate separately	PM
R7	Client expects 24/7 unattended monitoring from a web page	Medium	High	Clarify in Phase 0; offer desktop/server options	PM
R8	Privacy/legal concerns around camera use	Low–Medium	High	Local processing, consent text, camera-on indicator, optional legal review	PM/Client
R9	Unclear requirements or late changes	Medium	Medium	Weekly demos, written acceptance criteria, change log	PM
R10	Key team member unavailable	Low	Medium	Code reviews, documentation, shared knowledge	PM

10.1 Change Control

1. Change request submitted in writing (what, why, priority).
2. Team estimates effort and schedule impact.
3. Client approves or defers.
4. Approved changes are added to the backlog and the plan is updated.

11. Communication and Governance

Item	Frequency	Audience	Format
Sprint demo	Weekly (Fri)	Client, team	Live demo of working build
Status report	Weekly	Client sponsor	1-page summary: done, next, risks, decisions needed
Milestone review	At M0, M2, M4, M5	Client	Sign-off meeting
Daily check-in	Daily	Team	Chat or stand-up
Decision log	Ongoing	All	Shared document

Escalation path: Developer → Project Manager → Client Sponsor.

12. Deliverables

#	Deliverable	Phase
1	Requirements sign-off and wireframes	0
2	Source code repository with CI configured	0–5
3	MVP build on staging	2
4	Test plan, test cases and test report	5
5	Production deployment (Release 1.0)	5
6	User guide (how to use, tune sensitivity, troubleshoot camera)	5
7	Technical documentation (architecture, setup, deployment, configuration)	5
8	Release notes and handover session	5
9	(Optional) Backend service, API documentation and dashboard	6

13. Maintenance and Support (After Release)

Item	Plan
Warranty period	30 days: bug fixes at no extra cost
Bug fixing	Per severity table in Section 7.4
Browser updates	Quarterly check against new browser releases
Dependency updates	Monthly automated updates (Dependabot/Renovate) with tests
Monitoring	Optional error reporting (Sentry) and uptime check
Enhancements	Prioritized backlog; estimated and scheduled per request

14. Budget and Cost Considerations

Cost item	Notes
Development effort	~55–60 person-days for Release 1.0 (plus ~25 for the optional backend)
Hosting (Phase 1–5)	Very low: static hosting/CDN, often free to a few dollars per month
Domain and TLS	Domain yearly fee; TLS certificate free (Let's Encrypt/hosting provider)